SIMATS ENGINEERING
ASSESSMENT TOOL 3

COURSE NAME: Fundamentals of Data Science **COURSE CODE: DSA04**

COURSE OUTCOME COVERED

CO1: *Analyse the role of data science, facets of data, and the big data ecosystem for informed decision-making. (BL2)*

QUESTIONS: 05 **Total Marks:100**

Annexure A

Exploratory Data Analysis - Based Practical Assessment

Code of Conduct:

I S.MAGALAKSHMI (192524144) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

S. MAGALAKSHMI

Annexure A

Exploratory Data Analysis-Based Practical Assessment

1. Education Analytics Dataset

Question 1

A college wants to analyze student performance based on attendance, study hours, internal marks, and final result. Perform EDA to identify the factors affecting student results.

Student_ID	Attendance	Study_Hours	Internal_Marks	Result
S01	92	5	45	Pass
S02	55	2	24	Fail
S03	78	4	38	Pass
S04	60	3	28	Fail
S05	88	5	42	Pass
S06	47	1	20	Fail
S07	95	6	48	Pass
S08	72	3	35	Pass
S09	50	2	22	Fail
S10	83	4	40	Pass
S11	66	3	30	Fail
S12	90	5	44	Pass
S13	58	2	25	Fail
S14	76	4	36	Pass
S15	62	3	29	Fail
S16	85	5	41	Pass
S17	45	1	18	Fail
S18	80	4	39	Pass
S19	68	3	32	Pass
S20	53	2	23	Fail

2. Healthcare Analytics Dataset

Question 2

A hospital wants to analyze patient health risk based on age, blood sugar, blood pressure, BMI, and disease status. Perform EDA to find important health patterns.

Patient_ID	Age	Sugar_Level	BP	BMI	Disease_Status
P01	45	145	130	28	Yes
P02	32	98	118	23	No
P03	55	180	145	31	Yes
P04	28	90	110	22	No
P05	60	200	150	34	Yes
P06	40	120	125	26	No
P07	52	165	140	30	Yes
P08	35	105	120	24	No
P09	48	155	135	29	Yes
P10	30	95	115	23	No
P11	62	210	155	35	Yes
P12	38	115	122	25	No

P13	50	170	142	30	Yes
P14	27	88	108	21	No
P15	58	190	148	33	Yes
P16	42	130	128	27	No
P17	46	150	132	28	Yes
P18	34	100	116	24	No
P19	53	175	144	31	Yes
P20	29	92	112	22	No

3. Retail Sales Dataset
Question 3
A supermarket wants to analyze product sales based on category, price, quantity sold, discount, and revenue. Perform EDA to identify high-selling products and revenue patterns.

Product_ID	Category	Price	Quantity_Sold	Discount	Revenue
PR01	Grocery	50	20	5	1000
PR02	Snacks	30	35	3	1050
PR03	Dairy	45	18	2	810
PR04	Fruits	60	25	5	1500
PR05	Vegetables	40	30	4	1200
PR06	Grocery	80	15	6	1200
PR07	Snacks	25	40	3	1000
PR08	Dairy	55	16	2	880
PR09	Fruits	70	22	5	1540
PR10	Vegetables	35	28	4	980
PR11	Grocery	90	12	6	1080
PR12	Snacks	20	45	2	900
PR13	Dairy	65	14	3	910
PR14	Fruits	75	20	5	1500
PR15	Vegetables	45	26	4	1170
PR16	Grocery	100	10	7	1000
PR17	Snacks	35	32	3	1120
PR18	Dairy	50	19	2	950
PR19	Fruits	80	18	6	1440
PR20	Vegetables	30	35	3	1050

4. Banking Loan Approval Dataset
Question 4
A bank wants to analyze loan approval decisions based on income, credit score, loan amount, employment type, and approval status. Perform EDA to identify patterns related to loan approval.

Customer_ID	Income	Credit_Score	Loan_Amount	Employment_Type	Loan_Status
C01	45000	720	200000	Salaried	Approved
C02	25000	580	150000	Self-employed	Rejected
C03	60000	750	300000	Salaried	Approved
C04	22000	560	120000	Unemployed	Rejected
C05	50000	710	250000	Salaried	Approved
C06	30000	600	180000	Self-employed	Rejected

C07	70000	780	350000	Salaried	Approved
C08	28000	590	160000	Self-employed	Rejected
C09	55000	730	270000	Salaried	Approved
C10	24000	570	130000	Unemployed	Rejected
C11	65000	760	320000	Salaried	Approved
C12	35000	620	200000	Self-employed	Rejected
C13	48000	700	240000	Salaried	Approved
C14	26000	585	140000	Unemployed	Rejected
C15	75000	790	400000	Salaried	Approved
C16	32000	610	190000	Self-employed	Rejected
C17	52000	725	260000	Salaried	Approved
C18	27000	575	150000	Self-employed	Rejected
C19	68000	770	330000	Salaried	Approved
C20	23000	555	125000	Unemployed	Rejected

5. Agriculture Crop Yield Dataset

Question 5

An agricultural department wants to analyze crop yield based on rainfall, temperature, fertilizer usage, soil type, and crop yield. Perform EDA to identify the factors influencing crop production.

Farm_ID	Rainfall_mm	Temperature	Fertilizer_kg	Soil_Type	Crop_Yield_kg
F01	850	28	60	Loamy	3200
F02	600	32	45	Sandy	2100
F03	900	27	65	Loamy	3500
F04	500	34	40	Sandy	1800
F05	780	29	55	Clay	3000
F06	650	31	48	Sandy	2300
F07	920	26	70	Loamy	3700
F08	720	30	52	Clay	2800
F09	550	33	42	Sandy	1900
F10	800	28	58	Loamy	3100
F11	880	27	62	Loamy	3400
F12	620	32	46	Sandy	2200
F13	760	29	54	Clay	2950
F14	940	26	72	Loamy	3800
F15	580	33	44	Sandy	2000
F16	820	28	60	Clay	3150
F17	700	30	50	Clay	2700
F18	960	25	75	Loamy	3900
F19	540	34	41	Sandy	1850
F20	790	29	56	Clay	3050

Common EDA Tasks:

1. Identify the number of rows and columns.
2. Check data types of each column.
3. Find missing values and duplicate values.
4. Calculate mean, median, minimum, maximum, and standard deviation.
5. Draw suitable charts such as bar chart, histogram, scatter plot, and box plot.

6. Identify important patterns from the dataset.
7. Write 4 to 5 observations based on EDA.
8. Give a short conclusion from the analysis.

RUBRICS FOR CALCULATION

Exploratory Data Analysis-Based Practical Assessment					
Criteria	**Weightage**	**Level 4 – Exemplary**	**Level 3 – Proficient**	**Level 2 – Developing**	**Level 1 – Beginning**
Dataset Understanding	20	Clearly explains dataset source, features, target variables, data types, and problem context with strong clarity	Explains dataset, features, and variables with adequate clarity	Basic explanation of dataset with limited understanding of variables	Poor understanding of dataset; variables and problem context are unclear
Data Cleaning	25	Effectively identifies and handles missing values, duplicates, incorrect entries, and outliers using suitable methods	Handles most missing values, duplicates, and errors correctly	Performs basic cleaning but misses some important data quality issues	Minimal or incorrect data cleaning; major errors remain in the dataset
Data Analysis and Statistical Summary	20	Provides detailed descriptive statistics, comparisons, patterns, and meaningful interpretations	Provides relevant summary statistics and basic interpretation	Provides limited statistics with weak explanation	Lacks statistical analysis or gives incorrect interpretation
Data Visualization	20	Uses highly appropriate charts with proper labels, titles, and clear visual interpretation	Uses suitable charts with mostly clear labels and explanation	Uses limited or basic charts with unclear interpretation	Poor or incorrect visualizations; charts do not support analysis
Insight Generation and Reporting	15	Presents strong insights, conclusions, and recommendations based on data evidence	Presents clear findings and conclusions with reasonable support	Provides basic findings but lacks depth or proper justification	Findings are unclear, unsupported, or not related to the data

QUESTION 1
Input

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

# 1. Create dataset
df = pd.DataFrame({
    "Student_ID": [f"S{i:02d}" for i in range(1,21)],
    "Attendance": [92,55,78,60,88,47,95,72,50,83,66,90,58,76,62,85,45,80,68,53],
    "Study_Hours": [5,2,4,3,5,1,6,3,2,4,3,5,2,4,3,5,1,4,3,2],
    "Internal_Marks": [45,24,38,28,42,20,48,35,22,40,30,44,25,36,29,41,18,39,32,23],
    "Result": ["Pass","Fail","Pass","Fail","Pass","Fail","Pass","Pass","Fail","Pass",
               "Fail","Pass","Fail","Pass","Fail","Pass","Fail","Pass","Pass","Fail"]
})

print("EDUCATION ANALYTICS DATASET")
display(df)

# 2. Rows and columns
print("\nRows and columns:", df.shape)

# 3. Data types
print("\nData types:")
print(df.dtypes)

# 4. Missing and duplicate values
print("\nMissing values:")
print(df.isnull().sum())
print("Duplicate rows:", df.duplicated().sum())

# 5. Clean data
df = df.drop_duplicates().copy()
df = df.dropna(subset=["Result"])
for col in ["Attendance", "Study_Hours", "Internal_Marks"]:
    df[col] = df[col].fillna(df[col].median())

# 6. Statistical calculations
print("\nStatistical summary:")
display(df.describe().T)

numeric = df.select_dtypes(include=np.number)
print("\nMean:\n", numeric.mean())
print("\nMedian:\n", numeric.median())
print("\nMinimum:\n", numeric.min())
print("\nMaximum:\n", numeric.max())
print("\nStandard deviation:\n", numeric.std())
```

```
# 7. ONE SUITABLE CHART: Scatter plot
for result, group in df.groupby("Result"):
    plt.scatter(group["Attendance"], group["Internal_Marks"],
                label=result, s=80)

plt.title("Attendance vs Internal Marks by Student Result")
plt.xlabel("Attendance (%)")
plt.ylabel("Internal Marks")
plt.legend(title="Result")
plt.grid(True, alpha=0.3)
plt.show()

# 8. Observations
print("\nOBSERVATIONS")
print("1. Total students:", len(df))
print("2. Students passed:", (df["Result"] == "Pass").sum())
print("3. Students failed:", (df["Result"] == "Fail").sum())
print("4. Average attendance:", round(df["Attendance"].mean(), 2))
print("5. Average internal marks:", round(df["Internal_Marks"].mean(), 2))
print("\nAverage values by result:")
display(df.groupby("Result")[
    ["Attendance", "Study_Hours", "Internal_Marks"]
].mean().round(2))

# 9. Conclusion
print("\nCONCLUSION")
print("The dataset helps examine how attendance and internal marks")
print("vary with student results. The observations can help identify")
print("students who may need additional academic support.")
```

Output

EDUCATION ANALYTICS DATASET

	Student_ID	Attendance	Study_Hours	Internal_Marks	Result
0	S01	92	5	45	Pass
1	S02	55	2	24	Fail
2	S03	78	4	38	Pass
3	S04	60	3	28	Fail
4	S05	88	5	42	Pass
5	S06	47	1	20	Fail
6	S07	95	6	48	Pass
7	S08	72	3	35	Pass
8	S09	50	2	22	Fail
9	S10	83	4	40	Pass
10	S11	66	3	30	Fail
11	S12	90	5	44	Pass
12	S13	58	2	25	Fail
13	S14	76	4	36	Pass
14	S15	62	3	29	Fail
15	S16	85	5	41	Pass
16	S17	45	1	18	Fail
17	S18	80	4	39	Pass
18	S19	68	3	32	Pass
19	S20	53	2	23	Fail

Rows and columns: (20, 5)

Data types:
Student_ID object
Attendance int64
Study_Hours int64
Internal_Marks int64
Result object
dtype: object

Missing values:
Student_ID 0
Attendance 0
Study_Hours 0
Internal_Marks 0
Result 0
dtype: int64
Duplicate rows: 0

Statistical summary:

	count	mean	std	min	25%	50%	75%	max
Attendance	20.0	70.15	15.938286	45.0	57.25	70.0	83.50	95.0
Study_Hours	20.0	3.35	1.424411	1.0	2.00	3.0	4.25	6.0
Internal_Marks	20.0	32.95	9.087151	18.0	24.75	33.5	40.25	48.0

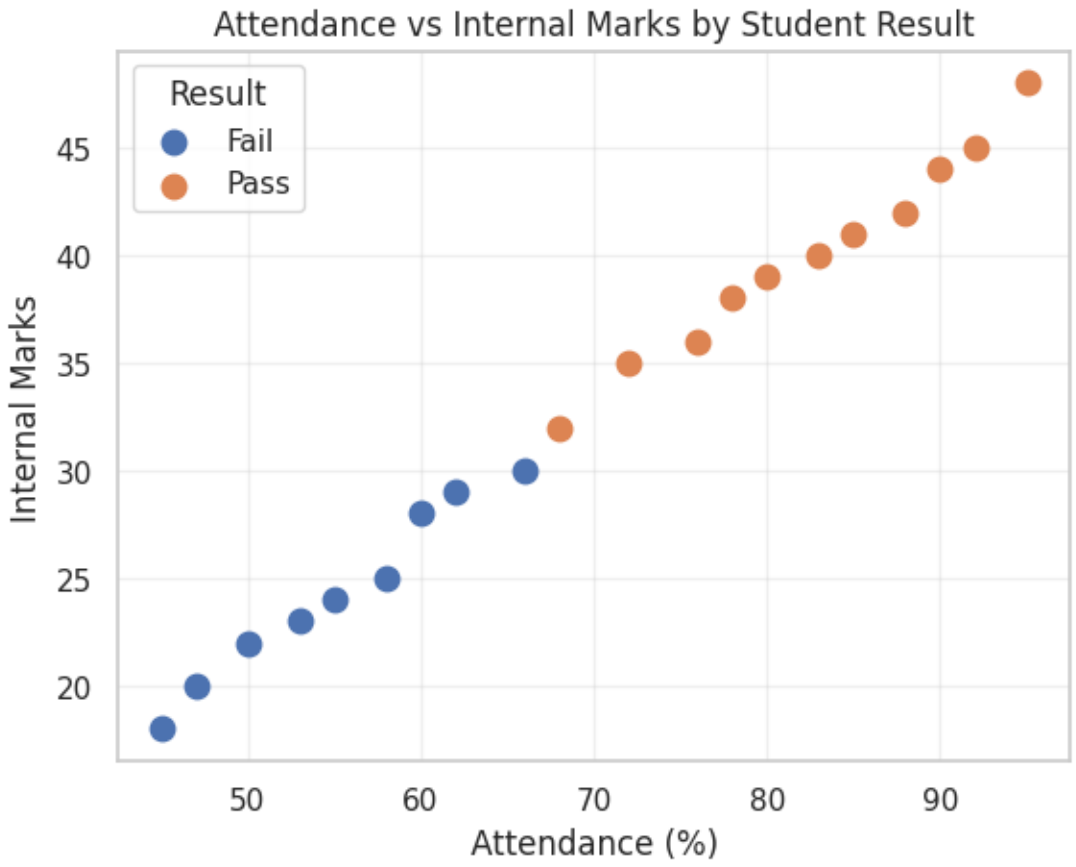
Mean:
Attendance 70.15
Study_Hours 3.35
Internal_Marks 32.95
dtype: float64

Median:
Attendance 70.0
Study_Hours 3.0
Internal_Marks 33.5
dtype: float64

Minimum:
Attendance 45
Study_Hours 1
Internal_Marks 18
dtype: int64

Maximum:
Attendance 95
Study_Hours 6
Internal_Marks 48
dtype: int64

Standard deviation:
Attendance 15.938286
Study_Hours 1.424411
Internal_Marks 9.087151
dtype: float64



OBSERVATIONS

1. Total students: 20
2. Students passed: 11
3. Students failed: 9
4. Average attendance: 70.15
5. Average internal marks: 32.95

Average values by result:

	Attendance	Study_Hours	Internal_Marks
Result			
Fail	55.11	2.11	24.33
Pass	82.45	4.36	40.00

CONCLUSION

The dataset helps examine how attendance and internal marks vary with student results. The observations can help identify students who may need additional academic support.

QUESTION 2
Input

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

# 1. Create dataset
df = pd.DataFrame({
    "Patient_ID": [f"P{i:02d}" for i in range(1,21)],
    "Age": [45,32,55,28,60,40,52,35,48,30,62,38,50,27,58,42,46,34,53,29],
    "Sugar_Level":
[145,98,180,90,200,120,165,105,155,95,210,115,170,88,190,130,150,100,175,92],
    "BP":
[130,118,145,110,150,125,140,120,135,115,155,122,142,108,148,128,132,116,144,112],
    "BMI": [28,23,31,22,34,26,30,24,29,23,35,25,30,21,33,27,28,24,31,22],
    "Disease_Status": ["Yes","No","Yes","No","Yes","No","Yes","No","Yes","No",
                       "Yes","No","Yes","No","Yes","No","Yes","No","Yes","No"]
})

print("HEALTHCARE ANALYTICS DATASET")
display(df)

# 2. Rows and columns
print("\nRows and columns:", df.shape)

# 3. Data types
print("\nData types:")
print(df.dtypes)

# 4. Missing and duplicate values
print("\nMissing values:")
print(df.isnull().sum())
print("Duplicate rows:", df.duplicated().sum())

# 5. Clean data
df = df.drop_duplicates().copy()
df = df.dropna(subset=["Disease_Status"])
for col in ["Age", "Sugar_Level", "BP", "BMI"]:
    df[col] = df[col].fillna(df[col].median())

# 6. Statistical calculations
numeric = df.select_dtypes(include=np.number)
print("\nStatistical summary:")
display(numeric.describe().T)
print("\nMean:\n", numeric.mean())
print("\nMedian:\n", numeric.median())
print("\nMinimum:\n", numeric.min())
print("\nMaximum:\n", numeric.max())
print("\nStandard deviation:\n", numeric.std())
```

```
# 7. ONE SUITABLE CHART: Scatter plot
for status, group in df.groupby("Disease_Status"):
    plt.scatter(group["Age"], group["Sugar_Level"],
                label=status, s=80)

plt.title("Age vs Sugar Level by Disease Status")
plt.xlabel("Age (years)")
plt.ylabel("Sugar Level")
plt.legend(title="Disease Status")
plt.grid(True, alpha=0.3)
plt.show()

# 8. Observations
print("\nOBSERVATIONS")
print("1. Total patients:", len(df))
print("2. Patients marked Yes:", (df["Disease_Status"] == "Yes").sum())
print("3. Patients marked No:", (df["Disease_Status"] == "No").sum())
print("4. Average age:", round(df["Age"].mean(), 2))
print("5. Average sugar level:", round(df["Sugar_Level"].mean(), 2))
print("\nAverage health measurements by disease status:")
display(df.groupby("Disease_Status")[
    ["Age", "Sugar_Level", "BP", "BMI"]
].mean().round(2))

# 9. Conclusion
print("\nCONCLUSION")
print("The analysis compares age and sugar level across disease-status")
print("groups and summarizes related health measurements. It describes")
print("the supplied data and is not a medical diagnosis.")
```

OUTPUT

HEALTHCARE ANALYTICS DATASET

	Patient_ID	Age	Sugar_Level	BP	BMI	Disease_Status
0	P01	45	145	130	28	Yes
1	P02	32	98	118	23	No
2	P03	55	180	145	31	Yes
3	P04	28	90	110	22	No
4	P05	60	200	150	34	Yes
5	P06	40	120	125	26	No
6	P07	52	165	140	30	Yes
7	P08	35	105	120	24	No
8	P09	48	155	135	29	Yes
9	P10	30	95	115	23	No
10	P11	62	210	155	35	Yes
11	P12	38	115	122	25	No
12	P13	50	170	142	30	Yes
13	P14	27	88	108	21	No
14	P15	58	190	148	33	Yes
15	P16	42	130	128	27	No
16	P17	46	150	132	28	Yes
17	P18	34	100	116	24	No
18	P19	53	175	144	31	Yes
19	P20	29	92	112	22	No

Rows and columns: (20, 6)

Data types:
Patient_ID object
Age int64
Sugar_Level int64
BP int64
BMI int64
Disease_Status object
dtype: object

Missing values:
Patient_ID 0
Age 0
Sugar_Level 0
BP 0
BMI 0
Disease_Status 0
dtype: int64

Duplicate rows: 0

Statistical summary:

	count	mean	std	min	25%	50%	75%	max
Age	20.0	43.20	11.316499	27.0	33.50	43.5	52.25	62.0
Sugar_Level	20.0	138.65	40.334490	88.0	99.50	137.5	171.25	210.0
BP	20.0	129.75	14.523574	108.0	117.50	129.0	142.50	155.0
BMI	20.0	27.30	4.256265	21.0	23.75	27.5	30.25	35.0

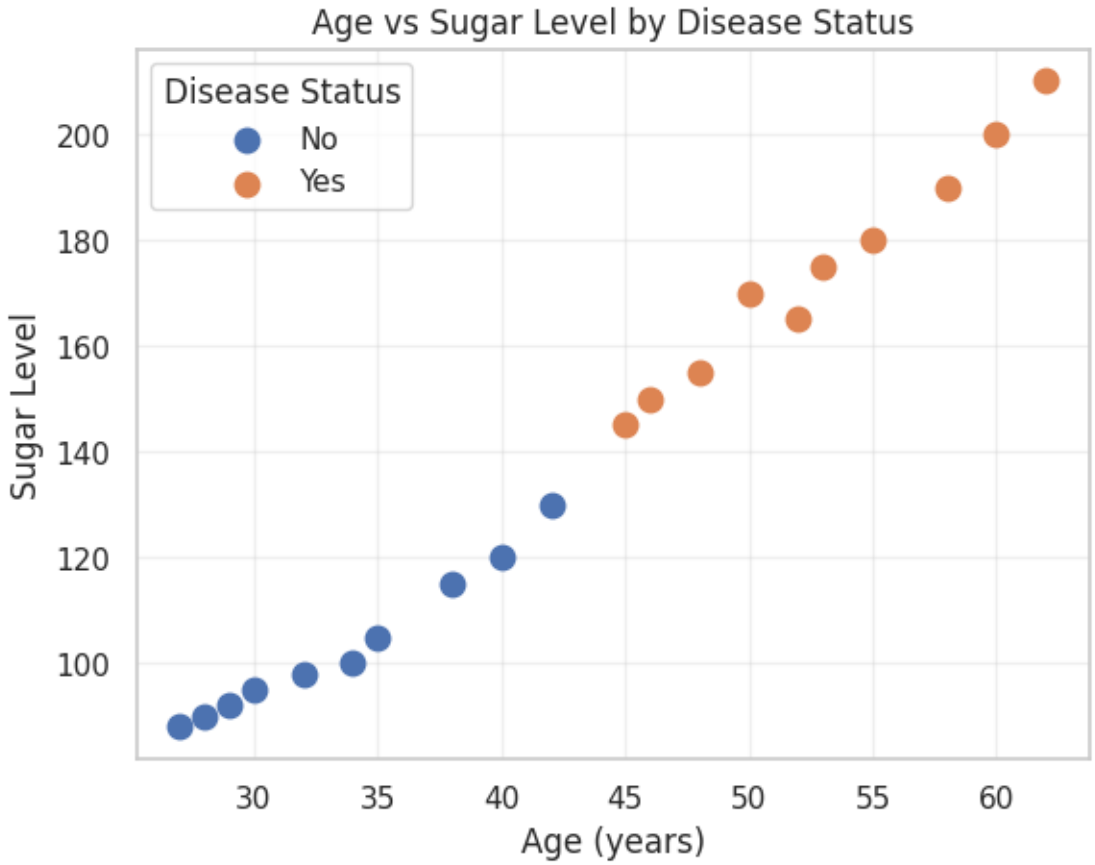
Mean:
Age 43.20
Sugar_Level 138.65
BP 129.75
BMI 27.30
dtype: float64

Median:
Age 43.5
Sugar_Level 137.5
BP 129.0
BMI 27.5
dtype: float64

Minimum:
Age 27
Sugar_Level 88
BP 108
BMI 21
dtype: int64

Maximum:
Age 62
Sugar_Level 210
BP 155
BMI 35
dtype: int64

Standard deviation:
Age 11.316499
Sugar_Level 40.334490
BP 14.523574
BMI 4.256265
dtype: float64



OBSERVATIONS
1. Total patients: 20
2. Patients marked Yes: 10
3. Patients marked No: 10
4. Average age: 43.2
5. Average sugar level: 138.65

Average health measurements by disease status:

	Age	Sugar_Level	BP	BMI
Disease_Status				
No	33.5	103.3	117.4	23.7
Yes	52.9	174.0	142.1	30.9

CONCLUSION
The analysis compares age and sugar level across disease-status groups and summarizes related health measurements. It describes the supplied data and is not a medical diagnosis.

QUESTION 3
Input

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

# 1. Create dataset
df = pd.DataFrame({
    "Product_ID": [f"PR{i:02d}" for i in range(1,21)],
    "Category": ["Grocery","Snacks","Dairy","Fruits","Vegetables"] * 4,
    "Price": [50,30,45,60,40,80,25,55,70,35,90,20,65,75,45,100,35,50,80,30],
    "Quantity_Sold": [20,35,18,25,30,15,40,16,22,28,12,45,14,20,26,10,32,19,18,35],
    "Discount": [5,3,2,5,4,6,3,2,5,4,6,2,3,5,4,7,3,2,6,3],
    "Revenue": [1000,1050,810,1500,1200,1200,1000,880,1540,980,
            1080,900,910,1500,1170,1000,1120,950,1440,1050]
})

print("RETAIL SALES DATASET")
display(df)

# 2. Rows and columns
print("\nRows and columns:", df.shape)

# 3. Data types
print("\nData types:")
print(df.dtypes)

# 4. Missing and duplicate values
print("\nMissing values:")
print(df.isnull().sum())
print("Duplicate rows:", df.duplicated().sum())

# 5. Clean data
df = df.drop_duplicates().copy()
for col in ["Price", "Quantity_Sold", "Discount", "Revenue"]:
    df[col] = df[col].fillna(df[col].median())
df = df.dropna(subset=["Category"])

# 6. Statistical calculations
numeric = df.select_dtypes(include=np.number)
print("\nStatistical summary:")
display(numeric.describe().T)
print("\nMean:\n", numeric.mean())
print("\nMedian:\n", numeric.median())
print("\nMinimum:\n", numeric.min())
print("\nMaximum:\n", numeric.max())
print("\nStandard deviation:\n", numeric.std())
```

```
# 7. ONE SUITABLE CHART: Revenue by category
category_revenue = df.groupby("Category")["Revenue"].sum().sort_values(ascending=False)

print("\nTotal revenue by category:")
display(category_revenue)

category_revenue.plot(kind="bar", figsize=(8,5), edgecolor="black")
plt.title("Total Revenue by Product Category")
plt.xlabel("Product Category")
plt.ylabel("Total Revenue")
plt.xticks(rotation=25)
plt.tight_layout()
plt.show()

# 8. Observations
print("\nOBSERVATIONS")
print("1. Total products:", len(df))
print("2. Total quantity sold:", df["Quantity_Sold"].sum())
print("3. Total revenue:", df["Revenue"].sum())
print("4. Average product revenue:", round(df["Revenue"].mean(), 2))
print("5. Highest-revenue product:", df.loc[df["Revenue"].idxmax(), "Product_ID"])
print("6. Highest-revenue category:", category_revenue.idxmax())

# 9. Conclusion
print("\nCONCLUSION")
print("The analysis compares revenue across product categories and")
print("identifies high-revenue categories. These findings can support")
print("retail sales and inventory planning.")
```

OUTPUT

RETAIL SALES DATASET

	Product_ID	Category	Price	Quantity_Sold	Discount	Revenue
0	PR01	Grocery	50	20	5	1000
1	PR02	Snacks	30	35	3	1050
2	PR03	Dairy	45	18	2	810
3	PR04	Fruits	60	25	5	1500
4	PR05	Vegetables	40	30	4	1200
5	PR06	Grocery	80	15	6	1200
6	PR07	Snacks	25	40	3	1000
7	PR08	Dairy	55	16	2	880
8	PR09	Fruits	70	22	5	1540
9	PR10	Vegetables	35	28	4	980
10	PR11	Grocery	90	12	6	1080
11	PR12	Snacks	20	45	2	900
12	PR13	Dairy	65	14	3	910
13	PR14	Fruits	75	20	5	1500
14	PR15	Vegetables	45	26	4	1170
15	PR16	Grocery	100	10	7	1000
16	PR17	Snacks	35	32	3	1120
17	PR18	Dairy	50	19	2	950
18	PR19	Fruits	80	18	6	1440
19	PR20	Vegetables	30	35	3	1050

Rows and columns: (20, 6)

Data types:
Product_ID object
Category object
Price int64
Quantity_Sold int64
Discount int64
Revenue int64
dtype: object

Missing values:
Product_ID 0
Category 0
Price 0
Quantity_Sold 0
Discount 0
Revenue 0
dtype: int64

Duplicate rows: 0

Statistical summary:

	count	mean	std	min	25%	50%	75%	max
Price	20.0	54.0	22.803509	20.0	35.0	50.0	71.25	100.0
Quantity_Sold	20.0	24.0	9.673621	10.0	17.5	21.0	30.50	45.0
Discount	20.0	4.0	1.555973	2.0	3.0	4.0	5.00	7.0
Revenue	20.0	1114.0	221.416493	810.0	972.5	1050.0	1200.00	1540.0

Mean:
Price 54.0
Quantity_Sold 24.0
Discount 4.0
Revenue 1114.0
dtype: float64

Median:
Price 50.0
Quantity_Sold 21.0
Discount 4.0
Revenue 1050.0
dtype: float64

Minimum:
Price 20
Quantity_Sold 10
Discount 2
Revenue 810
dtype: int64

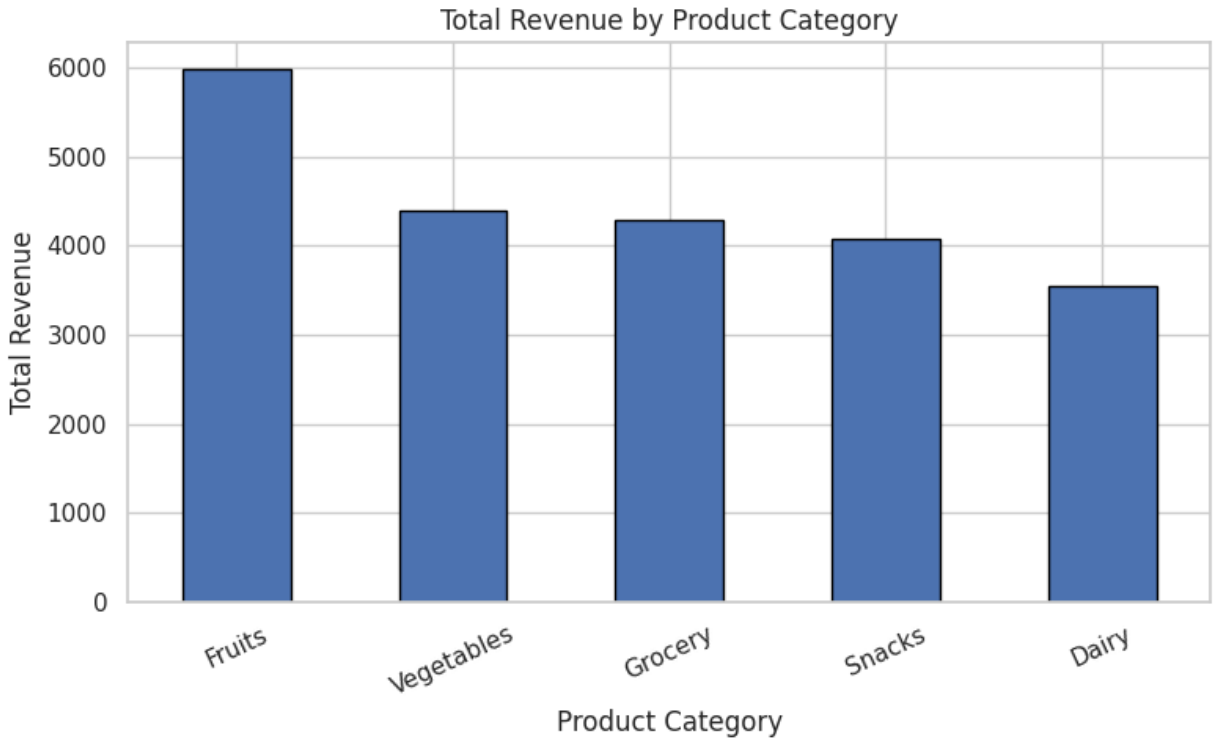
Maximum:
Price 100
Quantity_Sold 45
Discount 7
Revenue 1540
dtype: int64

Standard deviation:
Price 22.803509
Quantity_Sold 9.673621
Discount 1.555973
Revenue 221.416493
dtype: float64

Total revenue by category:

	Revenue
Category	
Fruits	5980
Vegetables	4400
Grocery	4280
Snacks	4070
Dairy	3550

dtype: int64



OBSERVATIONS
1. Total products: 20
2. Total quantity sold: 480
3. Total revenue: 22280
4. Average product revenue: 1114.0
5. Highest-revenue product: PR09
6. Highest-revenue category: Fruits

CONCLUSION
The analysis compares revenue across product categories and identifies high-revenue categories. These findings can support retail sales and inventory planning.

QUESTION 4
INPUT

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

# 1. Create dataset
df = pd.DataFrame({
   "Customer_ID": [f"C{i:02d}" for i in range(1,21)],
   "Income": [45000,25000,60000,22000,50000,30000,70000,28000,55000,24000,
           65000,35000,48000,26000,75000,32000,52000,27000,68000,23000],
   "Credit_Score":
[720,580,750,560,710,600,780,590,730,570,760,620,700,585,790,610,725,575,770,555],
   "Loan_Amount":
[200000,150000,300000,120000,250000,180000,350000,160000,270000,130000,
               320000,200000,240000,140000,400000,190000,260000,150000,330000,125000],
   "Employment_Type": ["Salaried","Self-employed","Salaried","Unemployed","Salaried",
                  "Self-employed","Salaried","Self-employed","Salaried","Unemployed",
                  "Salaried","Self-employed","Salaried","Unemployed","Salaried",
                  "Self-employed","Salaried","Self-employed","Salaried","Unemployed"],
   "Loan_Status": ["Approved","Rejected","Approved","Rejected","Approved","Rejected",
                "Approved","Rejected","Approved","Rejected","Approved","Rejected",
                "Approved","Rejected","Approved","Rejected","Approved","Rejected",
                "Approved","Rejected"]
})

print("BANKING LOAN APPROVAL DATASET")
display(df)

# 2. Rows and columns
print("\nRows and columns:", df.shape)

# 3. Data types
print("\nData types:")
print(df.dtypes)

# 4. Missing and duplicate values
print("\nMissing values:")
print(df.isnull().sum())
print("Duplicate rows:", df.duplicated().sum())

# 5. Clean data
df = df.drop_duplicates().copy()
df = df.dropna(subset=["Loan_Status"])
for col in ["Income", "Credit_Score", "Loan_Amount"]:
   df[col] = df[col].fillna(df[col].median())

# 6. Statistical calculations
```

```
numeric = df.select_dtypes(include=np.number)
print("\nStatistical summary:")
display(numeric.describe().T)
print("\nMean:\n", numeric.mean())
print("\nMedian:\n", numeric.median())
print("\nMinimum:\n", numeric.min())
print("\nMaximum:\n", numeric.max())
print("\nStandard deviation:\n", numeric.std())

# 7. ONE SUITABLE CHART: Income vs Credit Score
for status, group in df.groupby("Loan_Status"):
    plt.scatter(group["Income"], group["Credit_Score"],
                label=status, s=80)

plt.title("Income vs Credit Score by Loan Status")
plt.xlabel("Income")
plt.ylabel("Credit Score")
plt.legend(title="Loan Status")
plt.grid(True, alpha=0.3)
plt.show()

# 8. Observations
approved = df[df["Loan_Status"] == "Approved"]
rejected = df[df["Loan_Status"] == "Rejected"]

print("\nOBSERVATIONS")
print("1. Total customers:", len(df))
print("2. Approved loans:", len(approved))
print("3. Rejected loans:", len(rejected))
print("4. Approval percentage:",
      round((df["Loan_Status"] == "Approved").mean() * 100, 2), "%")
print("5. Average income of approved customers:", round(approved["Income"].mean(), 2))
print("6. Average income of rejected customers:", round(rejected["Income"].mean(), 2))
print("7. Average credit score of approved customers:",
round(approved["Credit_Score"].mean(), 2))
print("8. Average credit score of rejected customers:", round(rejected["Credit_Score"].mean(),
2))

# 9. Conclusion
print("\nCONCLUSION")
print("The analysis compares income and credit scores across loan")
print("statuses to identify patterns in the supplied customer data.")
print("These patterns do not establish the bank's actual approval rules.")
```

OUTPUT

BANKING LOAN APPROVAL DATASET

	Customer_ID	Income	Credit_Score	Loan_Amount	Employment_Type	Loan_Status
0	C01	45000	720	200000	Salaried	Approved
1	C02	25000	580	150000	Self-employed	Rejected
2	C03	60000	750	300000	Salaried	Approved
3	C04	22000	560	120000	Unemployed	Rejected
4	C05	50000	710	250000	Salaried	Approved
5	C06	30000	600	180000	Self-employed	Rejected
6	C07	70000	780	350000	Salaried	Approved
7	C08	28000	590	160000	Self-employed	Rejected
8	C09	55000	730	270000	Salaried	Approved
9	C10	24000	570	130000	Unemployed	Rejected
10	C11	65000	760	320000	Salaried	Approved
11	C12	35000	620	200000	Self-employed	Rejected
12	C13	48000	700	240000	Salaried	Approved
13	C14	26000	585	140000	Unemployed	Rejected
14	C15	75000	790	400000	Salaried	Approved
15	C16	32000	610	190000	Self-employed	Rejected
16	C17	52000	725	260000	Salaried	Approved
17	C18	27000	575	150000	Self-employed	Rejected
18	C19	68000	770	330000	Salaried	Approved
19	C20	23000	555	125000	Unemployed	Rejected

Rows and columns: (20, 6)

Data types:
Customer_ID object
Income int64
Credit_Score int64
Loan_Amount int64
Employment_Type object
Loan_Status object
dtype: object

Missing values:
Customer_ID 0
Income 0
Credit_Score 0
Loan_Amount 0
Employment_Type 0
Loan_Status 0
dtype: int64

Duplicate rows: 0

Statistical summary:

	count	mean	std	min	25%	50%	75%	max
Income	20.0	43000.0	17923.815383	22000.0	26750.00	40000.0	56250.0	75000.0
Credit_Score	20.0	664.0	85.526235	555.0	583.75	660.0	735.0	790.0
Loan_Amount	20.0	223250.0	83733.207272	120000.0	150000.00	200000.0	277500.0	400000.0

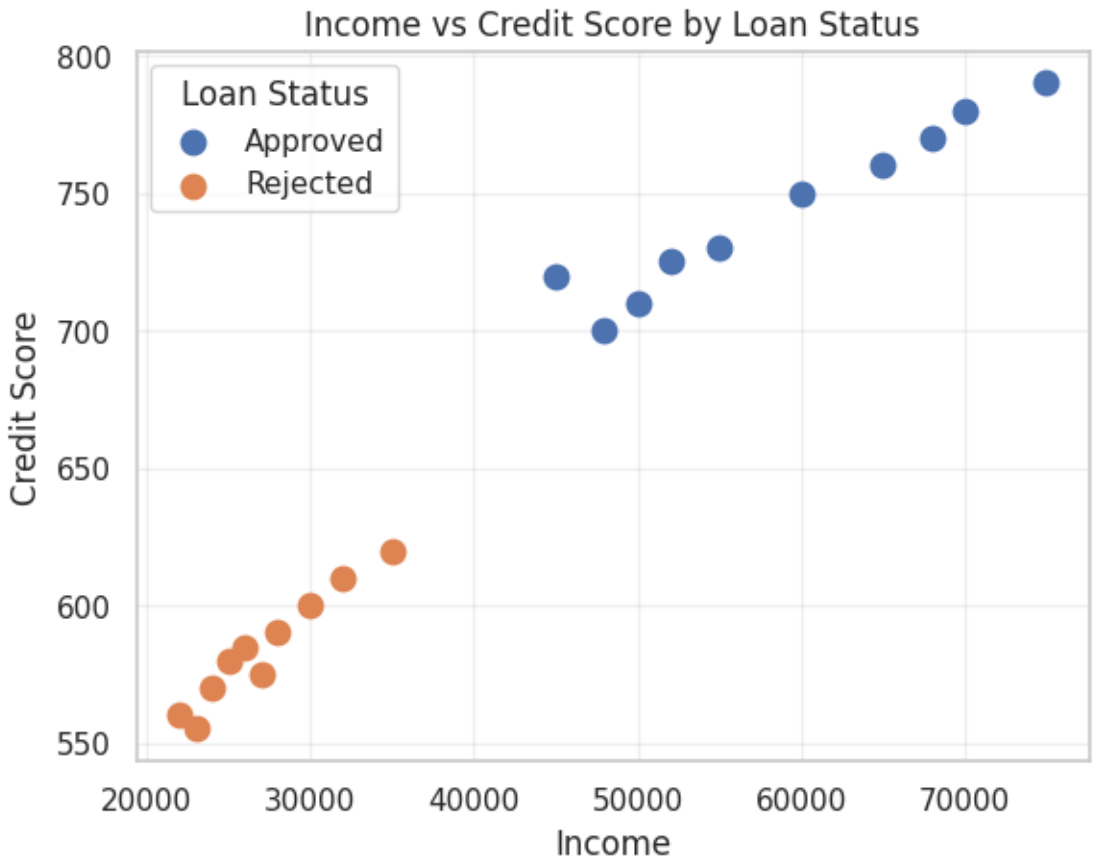
Mean:
```
Income          43000.0
Credit_Score      664.0
Loan_Amount    223250.0
dtype: float64
```

Median:
```
Income          40000.0
Credit_Score      660.0
Loan_Amount    200000.0
dtype: float64
```

Minimum:
```
Income          22000
Credit_Score      555
Loan_Amount    120000
dtype: int64
```

Maximum:
```
Income          75000
Credit_Score      790
Loan_Amount    400000
dtype: int64
```

Standard deviation:
```
Income          17923.815383
Credit_Score       85.526235
Loan_Amount     83733.207272
dtype: float64
```



OBSERVATIONS

1. Total customers: 20
2. Approved loans: 10
3. Rejected loans: 10
4. Approval percentage: 50.0 %
5. Average income of approved customers: 58800.0
6. Average income of rejected customers: 27200.0
7. Average credit score of approved customers: 743.5
8. Average credit score of rejected customers: 584.5

CONCLUSION

The analysis compares income and credit scores across loan statuses to identify patterns in the supplied customer data. These patterns do not establish the bank's actual approval rules.

QUESTION 5
INPUT

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

# 1. Create dataset
df = pd.DataFrame({
   "Farm_ID": [f"F{i:02d}" for i in range(1,21)],
   "Rainfall_mm":
[850,600,900,500,780,650,920,720,550,800,880,620,760,940,580,820,700,960,540,790],
   "Temperature": [28,32,27,34,29,31,26,30,33,28,27,32,29,26,33,28,30,25,34,29],
   "Fertilizer_kg": [60,45,65,40,55,48,70,52,42,58,62,46,54,72,44,60,50,75,41,56],
   "Soil_Type":
["Loamy","Sandy","Loamy","Sandy","Clay","Sandy","Loamy","Clay","Sandy","Loamy",

"Loamy","Sandy","Clay","Loamy","Sandy","Clay","Clay","Loamy","Sandy","Clay"],
   "Crop_Yield_kg": [3200,2100,3500,1800,3000,2300,3700,2800,1900,3100,
                     3400,2200,2950,3800,2000,3150,2700,3900,1850,3050]
})

print("AGRICULTURE CROP YIELD DATASET")
display(df)

# 2. Rows and columns
print("\nRows and columns:", df.shape)

# 3. Data types
print("\nData types:")
print(df.dtypes)

# 4. Missing and duplicate values
print("\nMissing values:")
print(df.isnull().sum())
print("Duplicate rows:", df.duplicated().sum())

# 5. Clean data
df = df.drop_duplicates().copy()
for col in ["Rainfall_mm", "Temperature", "Fertilizer_kg", "Crop_Yield_kg"]:
   df[col] = df[col].fillna(df[col].median())
df = df.dropna(subset=["Soil_Type"])

# 6. Statistical calculations
numeric = df.select_dtypes(include=np.number)
print("\nStatistical summary:")
display(numeric.describe().T)
print("\nMean:\n", numeric.mean())
print("\nMedian:\n", numeric.median())
print("\nMinimum:\n", numeric.min())
```

```
print("\nMaximum:\n", numeric.max())
print("\nStandard deviation:\n", numeric.std())

# 7. ONE SUITABLE CHART: Rainfall vs Crop Yield
for soil, group in df.groupby("Soil_Type"):
    plt.scatter(group["Rainfall_mm"], group["Crop_Yield_kg"],
                label=soil, s=80)

plt.title("Rainfall vs Crop Yield by Soil Type")
plt.xlabel("Rainfall (mm)")
plt.ylabel("Crop Yield (kg)")
plt.legend(title="Soil Type")
plt.grid(True, alpha=0.3)
plt.show()

# 8. Observations
print("\nOBSERVATIONS")
print("1. Total farms:", len(df))
print("2. Average rainfall:", round(df["Rainfall_mm"].mean(), 2), "mm")
print("3. Average temperature:", round(df["Temperature"].mean(), 2))
print("4. Average fertilizer usage:", round(df["Fertilizer_kg"].mean(), 2), "kg")
print("5. Average crop yield:", round(df["Crop_Yield_kg"].mean(), 2), "kg")
print("6. Highest crop yield:", df["Crop_Yield_kg"].max(), "kg")
print("7. Soil type with highest average yield:",
      df.groupby("Soil_Type")["Crop_Yield_kg"].mean().idxmax())

print("\nAverage yield by soil type:")
display(df.groupby("Soil_Type")["Crop_Yield_kg"].mean().round(2))

# 9. Conclusion
print("\nCONCLUSION")
print("The analysis examines the relationship between rainfall and crop")
print("yield while distinguishing soil types. It helps identify patterns")
print("for further agricultural investigation.")
```

OUTPUT

AGRICULTURE CROP YIELD DATASET

	Farm_ID	Rainfall_mm	Temperature	Fertilizer_kg	Soil_Type	Crop_Yield_kg
0	F01	850	28	60	Loamy	3200
1	F02	600	32	45	Sandy	2100
2	F03	900	27	65	Loamy	3500
3	F04	500	34	40	Sandy	1800
4	F05	780	29	55	Clay	3000
5	F06	650	31	48	Sandy	2300
6	F07	920	26	70	Loamy	3700
7	F08	720	30	52	Clay	2800
8	F09	550	33	42	Sandy	1900
9	F10	800	28	58	Loamy	3100
10	F11	880	27	62	Loamy	3400
11	F12	620	32	46	Sandy	2200
12	F13	760	29	54	Clay	2950
13	F14	940	26	72	Loamy	3800
14	F15	580	33	44	Sandy	2000
15	F16	820	28	60	Clay	3150
16	F17	700	30	50	Clay	2700
17	F18	960	25	75	Loamy	3900
18	F19	540	34	41	Sandy	1850
19	F20	790	29	56	Clay	3050

Rows and columns: (20, 6)

Data types:
Farm_ID object
Rainfall_mm int64
Temperature int64
Fertilizer_kg int64
Soil_Type object
Crop_Yield_kg int64
dtype: object

Missing values:
Farm_ID 0
Rainfall_mm 0
Temperature 0
Fertilizer_kg 0
Soil_Type 0
Crop_Yield_kg 0
dtype: int64

Duplicate rows: 0

Statistical summary:

	count	mean	std	min	25%	50%	75%	max
Rainfall_mm	20.0	743.00	144.335136	500.0	615.00	770.0	857.5	960.0
Temperature	20.0	29.55	2.762055	25.0	27.75	29.0	32.0	34.0
Fertilizer_kg	20.0	54.75	10.507516	40.0	45.75	54.5	60.5	75.0
Crop_Yield_kg	20.0	2820.00	681.407213	1800.0	2175.00	2975.0	3250.0	3900.0

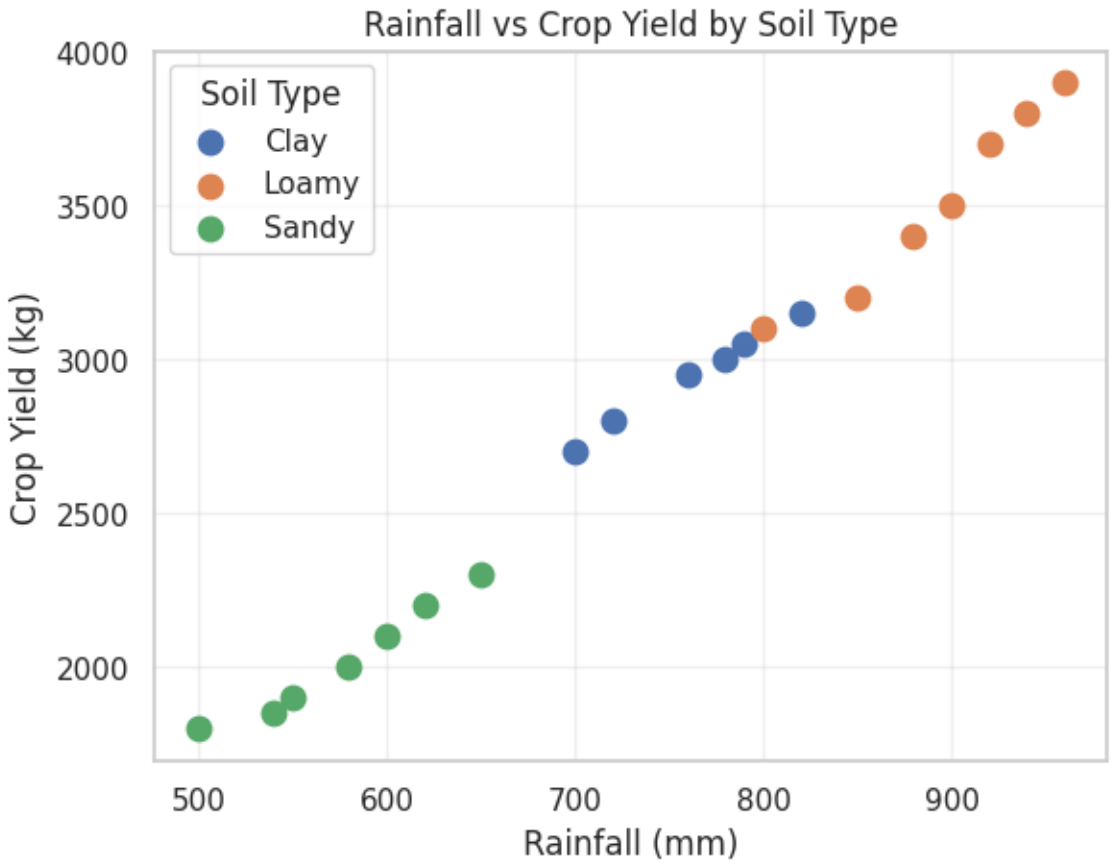
Mean:
Rainfall_mm 743.00
Temperature 29.55
Fertilizer_kg 54.75
Crop_Yield_kg 2820.00
dtype: float64

Median:
Rainfall_mm 770.0
Temperature 29.0
Fertilizer_kg 54.5
Crop_Yield_kg 2975.0
dtype: float64

Minimum:
Rainfall_mm 500
Temperature 25
Fertilizer_kg 40
Crop_Yield_kg 1800
dtype: int64

Maximum:
Rainfall_mm 960
Temperature 34
Fertilizer_kg 75
Crop_Yield_kg 3900
dtype: int64

Standard deviation:
Rainfall_mm 144.335136
Temperature 2.762055
Fertilizer_kg 10.507516
Crop_Yield_kg 681.407213
dtype: float64



OBSERVATIONS
1. Total farms: 20
2. Average rainfall: 743.0 mm
3. Average temperature: 29.55
4. Average fertilizer usage: 54.75 kg
5. Average crop yield: 2820.0 kg
6. Highest crop yield: 3900 kg
7. Soil type with highest average yield: Loamy

Average yield by soil type:

	Crop_Yield_kg
Soil_Type	
Clay	2941.67
Loamy	3514.29
Sandy	2021.43

dtype: float64
CONCLUSION
The analysis examines the relationship between rainfall and crop yield while distinguishing soil types. It helps identify patterns for further agricultural investigation.